

CS-590 Database Design and Development

6-1 Assignment: Graph Databases and Neo4j

Malgorzata Debska

Southern New Hampshire University

Zach Probst

March 10, 2026

Comparison of Relational, Document, and Graph Databases

In this assignment, Neo4j was used to explore how nutrient data from the Child Nutrition database could be modeled within a graph database. The assignment involved comparing relational, document, and graph databases, creating a new user within the Neo4j environment, implementing a schema using nodes and relationships, performing ETL processes on nutrient datasets, and executing Cypher queries to analyze the stored data.

Differences in Data Modeling

Relational databases store data in tables consisting of rows and columns. Each table is an entity, and relationships between entities are supported through foreign keys and primary keys.

Structured Query Language (SQL) is typically used to retrieve and manipulate relational data.

When information must be retrieved across multiple tables, relational databases rely on JOIN operations. Document databases store data in flexible document structures, usually formatted as JSON or BSON. Instead of organizing data into tables, document databases store records as self-contained documents that can include nested fields and arrays. This flexible structure allows developers to easily change data structures without redesigning the entire database schema.

Graph databases stand for data using nodes, relationships, and properties. Nodes are entities; relationships define connections between nodes, and properties store attributes associated with nodes or relationships. Queries in graph databases use languages such as Cypher to traverse connections between nodes rather than performing joins across tables.

Strengths and Weaknesses

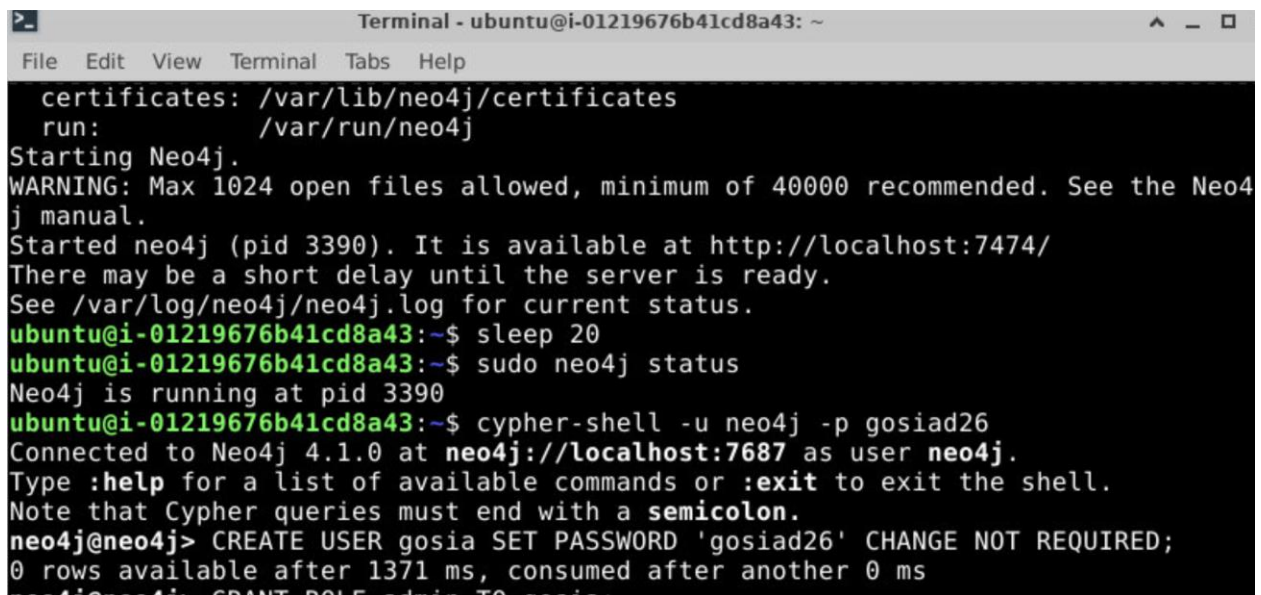
Relational databases are highly structured and provide strong data integrity and consistency. They are widely used in financial systems, enterprise resource planning systems, and applications that require reliable transaction processing. However, relational databases may become inefficient when managing highly connected data because queries often require multiple joins across tables. Document databases provide flexibility for applications where data structures often change. They are commonly used in web applications, content management systems, and systems that require scalable storage of semi-structured data. However, document databases may struggle when complex relationships between documents must be analyzed. Graph databases excel at storing and analyzing complex relationships. Because relationships are stored directly in the database, queries can quickly traverse connections between nodes. Graph databases are commonly used in social networks, fraud detection systems, and recommendation engines. A limitation of graph databases is that they may be less efficient for simple transactional systems where relational databases are already improved.

Creating a New User in Neo4j

The first step in the assignment involved creating a new user within the Neo4j database environment. This step ensures secure access to the database and allows administrative control over database operations. Using the Cypher shell, a connection was set up to the Neo4j server. The following command was used to create a new user: `CREATE USER gosia SET`

PASSWORD 'gosiad26' CHANGE NOT REQUIRED; This command created a new user account with a password that allows the user to log in and access the Neo4j database instance.

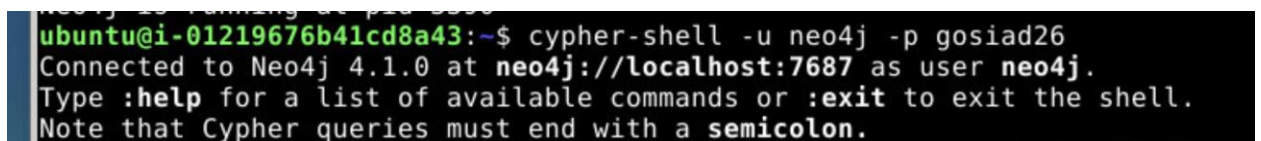
Screenshot – Creating New User in Neo4j

A terminal window titled 'Terminal - ubuntu@i-01219676b41cd8a43: ~' showing the process of starting Neo4j and creating a new user. The terminal output includes the configuration of certificates and run directories, starting the Neo4j service with a warning about open files, and then checking the status of the running service. Finally, a Cypher shell is used to create a new user named 'gosia' with the password 'gosiad26'.

```
certificates: /var/lib/neo4j/certificates
run: /var/run/neo4j
Starting Neo4j.
WARNING: Max 1024 open files allowed, minimum of 40000 recommended. See the Neo4j manual.
Started neo4j (pid 3390). It is available at http://localhost:7474/
There may be a short delay until the server is ready.
See /var/log/neo4j/neo4j.log for current status.
ubuntu@i-01219676b41cd8a43:~$ sleep 20
ubuntu@i-01219676b41cd8a43:~$ sudo neo4j status
Neo4j is running at pid 3390
ubuntu@i-01219676b41cd8a43:~$ cypher-shell -u neo4j -p gosiad26
Connected to Neo4j 4.1.0 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> CREATE USER gosia SET PASSWORD 'gosiad26' CHANGE NOT REQUIRED;
0 rows available after 1371 ms, consumed after another 0 ms
neo4j@neo4j>
```

Instantiating the Neo4j Database

After creating the user account, the Neo4j database was started and verified using the following command: `sudo neo4j status` The output confirmed that the Neo4j database server was running and ready to accept connections.

A terminal window showing the connection to the Neo4j database using the Cypher shell. The user 'neo4j' is connected with the password 'gosiad26'. The terminal output shows the connection details and the help message.

```
ubuntu@i-01219676b41cd8a43:~$ cypher-shell -u neo4j -p gosiad26
Connected to Neo4j 4.1.0 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
```

Implementing the Schema Using Nodes and Relationships

The schema for the graph database was implemented using two types of nodes:

NutDes (Nutrient Description Nodes)

These nodes are nutrient descriptions and include the following properties:

- NutrientCode
- NutrientDescription
- NutrientDescriptionAbbrev
- NutrientUnit
- DateAdded
- LastModified

The following Cypher query was used to create these nodes from the dataset.

```
LOAD CSV WITH HEADERS FROM "file:///Module_Six_Nut_Descs.csv" AS row
CREATE (n:NutDes)
SET n.NutrientCode = toInteger(row.`Nutrient code`),
n.NutrientDescription = row.`Nutrient description`,
n.NutrientDescriptionAbbrev = row.`Nutrient description abbrev`,
n.NutrientUnit = row.`Nutrient unit`,
n.DateAdded = row.`Date added`,
n.LastModified = row.`Last modified`;
```

Screenshot – Creating Nutrient Description Nodes

```
(Failure when processing file /var/lib/neo4j/import/Module_Six_Nut_Descs.csv on line 2.)
neo4j@neo4j> LOAD CSV WITH HEADERS from "file:///Module_Six_Nut_Descs.csv" AS row
                CREATE (n:NutDes)
                SET n.NutrientCode = toInteger(row.`Nutrient code`),
                    n.NutrientDescription = row.`Nutrient description`,
                    n.NutrientDescriptionAbbrev = row.`Nutrient description abbrev`,
                    n.NutrientUnit = row.`Nutrient unit`,
                    n.DateAdded = row.`Date added`,
                    n.LastModified = row.`Last modified`;
0 rows available after 655 ms, consumed after another 0 ms
Added 19 nodes, Set 114 properties, Added 19 labels
neo4j@neo4j> █
```

Extract, Transform, and Load (ETL) Process

The ETL process moved data from CSV files into the Neo4j database. ETL stands for extract, transform, and load.

Extract

The nutrient datasets were extracted from CSV files found in the Neo4j import directory:

- Module_Six_Nut_Descs.csv
- Module_Six_Nut_Vals.csv

These files hold information about nutrient descriptions and values.

Transform

During the transformation stage, data types were converted to be safely stored in the graph database. For example:

- Nutrient codes were converted to integers using `toInteger()`
- Nutrient values were converted to floating-point numbers using `toFloat()`

This ensured that the data was stored correctly in Neo4j.

Load

The transformed data was loaded into the database as nodes using Cypher queries.

```
LOAD CSV WITH HEADERS FROM "file:///Module_Six_Nut_Vals.csv" AS rec

CREATE (nv:NutVal)

SET nv.CNCode = toInteger(rec.`Cn Code`),

nv.NutrientCode = toInteger(rec.`Nutrient code`),

nv.NutrientValue = toFloat(rec.`Nutrient value`),

nv.PerUnit = rec.`Per unit`,

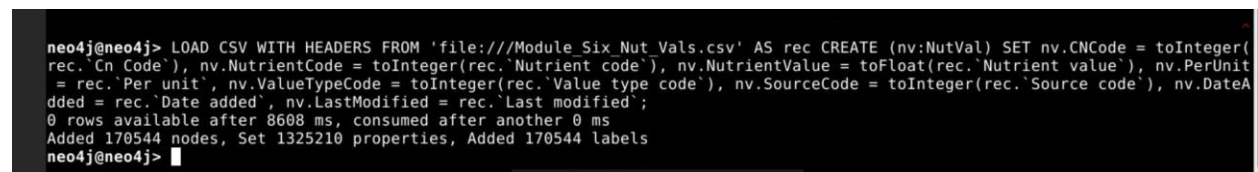
nv.ValueTypeCode = toInteger(rec.`Value type code`),

nv.SourceCode = toInteger(rec.`Source code`),

nv.DateAdded = rec.`Date added`,

nv.LastModified = rec.`Last modified`;
```

Screenshot – Loading Nutrient Value Data



```
neo4j@neo4j> LOAD CSV WITH HEADERS FROM 'file:///Module_Six_Nut_Vals.csv' AS rec CREATE (nv:NutVal) SET nv.CNCode = toInteger(
rec.`Cn Code`), nv.NutrientCode = toInteger(rec.`Nutrient code`), nv.NutrientValue = toFloat(rec.`Nutrient value`), nv.PerUnit
= rec.`Per unit`, nv.ValueTypeCode = toInteger(rec.`Value type code`), nv.SourceCode = toInteger(rec.`Source code`), nv.DateA
dded = rec.`Date added`, nv.LastModified = rec.`Last modified`;
0 rows available after 8608 ms, consumed after another 0 ms
Added 170544 nodes, Set 1325210 properties, Added 170544 labels
neo4j@neo4j> █
```

Creating Relationships Between Nodes

After both datasets were loaded into Neo4j, relationships were created between the nodes based on the shared NutrientCode attribute.

```

MATCH (nv:NutVal), (n:NutDes)
WHERE nv.NutrientCode = n.NutrientCode
CREATE (nv)-[:INSTANCE_OF]->(n)
RETURN nv, n;

```

This query created a relationship labeled **INSTANCE_OF** between each nutrient value node and the corresponding nutrient description node.

Screenshot – Relationships Created

```
| (:NutVal {LastModified: "01/23/2025", SourceCode: 3, CNCode: 125485, NutrientV
alue: 73.0, ValueTypeCode: 6, PerUnit: "100g", NutrientCode: 601, DateAdded: "01
/23/2025"}) | (:NutDes {LastModified: "10/29/1997", NutrientDescriptionAbbrev: "
Chol", NutrientDescription: "Cholesterol", NutrientCode: 601, NutrientUnit: "mg"
, DateAdded: "10/29/1997"})
| (:NutVal {LastModified: "01/23/2025", SourceCode: 3, CNCode: 125485, NutrientV
alue: 0.0, ValueTypeCode: 6, PerUnit: "100g", NutrientCode: 605, DateAdded: "01/
23/2025"}) | (:NutDes {LastModified: "11/24/2005", NutrientDescriptionAbbrev: "
Trans", NutrientDescription: "Total Trans", NutrientCode: 605, DateAdded: "11/24
/2005", NutrientUnit: "g"})
| (:NutVal {LastModified: "01/23/2025", SourceCode: 3, CNCode: 125485, NutrientV
alue: 7.32, ValueTypeCode: 6, PerUnit: "100g", NutrientCode: 606, DateAdded: "01
/23/2025"}) | (:NutDes {LastModified: "10/29/1997", NutrientDescriptionAbbrev: "
Sfat", NutrientDescription: "Saturated Fat", NutrientCode: 606, NutrientUnit: "g
", DateAdded: "10/29/1997"})
+-----+
-----
-----
-----
-----+

170544 rows available after 990 ms, consumed after another 11 ms
Created 170544 relationships
neo4j@neo4j>
```

Executing Cypher Queries

Several Cypher queries were executed to confirm that the data and relationships were correctly stored in the graph database.

Query 1

```
MATCH (n:NutDes)
```

```
RETURN n
```

```
LIMIT 10;
```

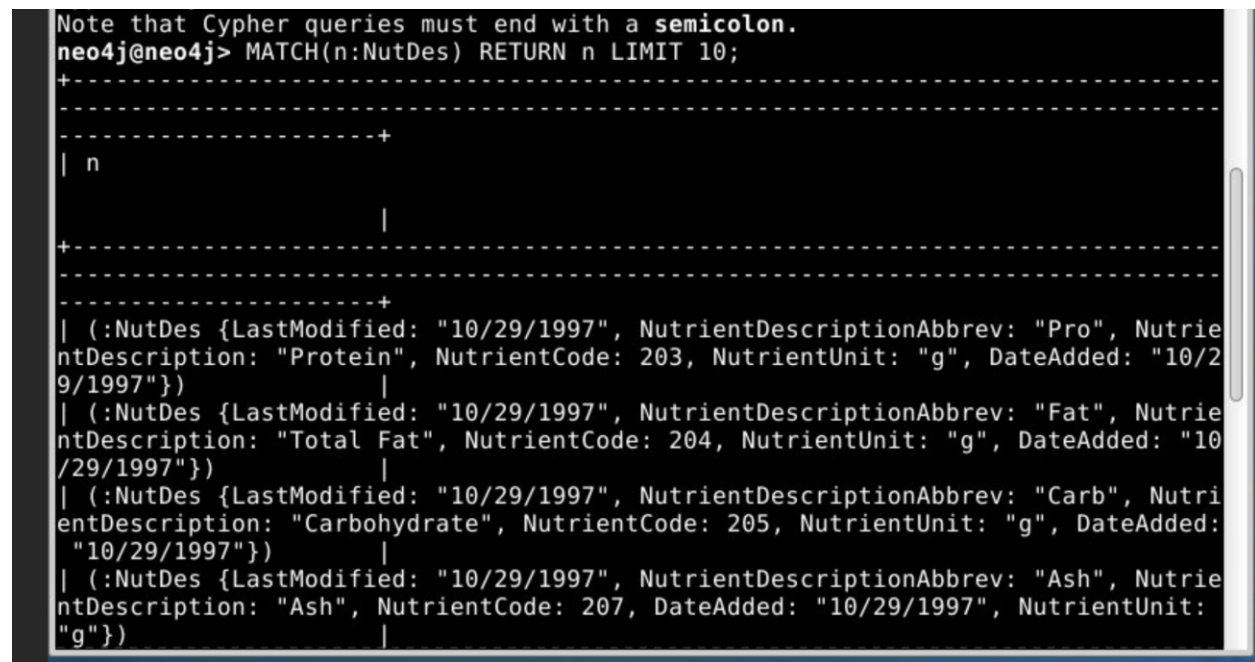
What the query does:

This query retrieves nodes labeled NutDes from the database.

What the query returns:

The query returns the first 10 nutrient description nodes stored in the graph database.

Screenshot – Query Results for Nutrient Descriptions



```
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> MATCH(n:NutDes) RETURN n LIMIT 10;
+-----+
| n
+-----+
| (:NutDes {LastModified: "10/29/1997", NutrientDescriptionAbbrev: "Pro", NutrientDescription: "Protein", NutrientCode: 203, NutrientUnit: "g", DateAdded: "10/29/1997"})
| (:NutDes {LastModified: "10/29/1997", NutrientDescriptionAbbrev: "Fat", NutrientDescription: "Total Fat", NutrientCode: 204, NutrientUnit: "g", DateAdded: "10/29/1997"})
| (:NutDes {LastModified: "10/29/1997", NutrientDescriptionAbbrev: "Carb", NutrientDescription: "Carbohydrate", NutrientCode: 205, NutrientUnit: "g", DateAdded: "10/29/1997"})
| (:NutDes {LastModified: "10/29/1997", NutrientDescriptionAbbrev: "Ash", NutrientDescription: "Ash", NutrientCode: 207, DateAdded: "10/29/1997", NutrientUnit: "g"})
```

Query 2

```
MATCH (nv:NutVal)
```

```
RETURN nv
```

```
LIMIT 10;
```

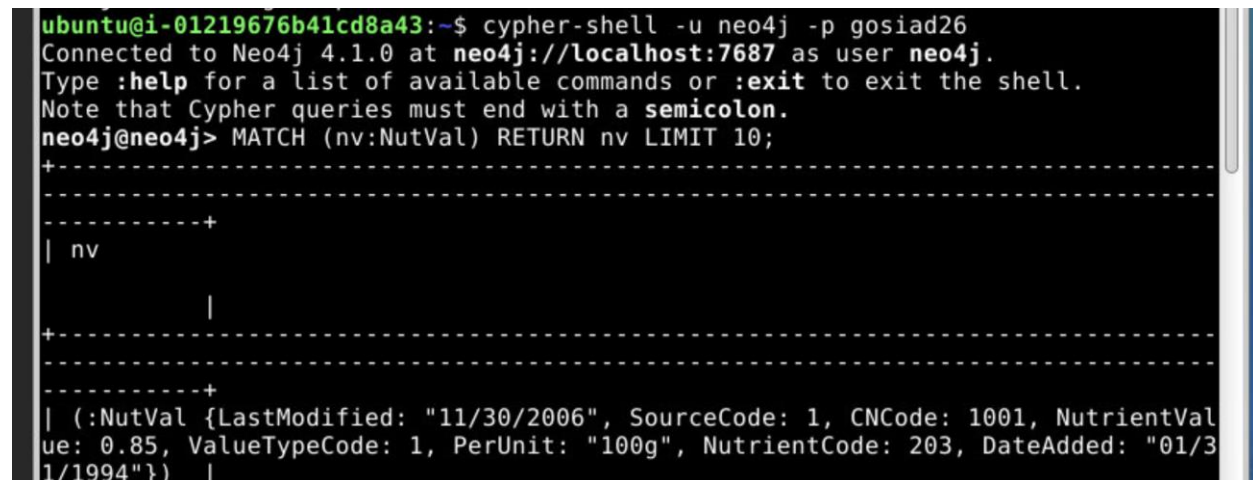
What the query does:

This query retrieves nodes labeled NutVal.

What the query returns:

The query returns the first 10 nutrient value records stored in the graph database.

Screenshot – Query Results for Nutrient Values



```
ubuntu@i-01219676b41cd8a43:~$ cypher-shell -u neo4j -p gosiad26
Connected to Neo4j 4.1.0 at neo4j://localhost:7687 as user neo4j.
Type :help for a list of available commands or :exit to exit the shell.
Note that Cypher queries must end with a semicolon.
neo4j@neo4j> MATCH (nv:NutVal) RETURN nv LIMIT 10;
+-----+
| nv |
+-----+
| (:NutVal {LastModified: "11/30/2006", SourceCode: 1, CNCode: 1001, NutrientValue: 0.85, ValueTypeCode: 1, PerUnit: "100g", NutrientCode: 203, DateAdded: "01/31/1994"}) |
+-----+
```

Query 3

```
MATCH (nv:NutVal)-[r:INSTANCE_OF]->(n:NutDes)
```

```
RETURN nv, r, n
```

```
LIMIT 20;
```

What the query does: This query searches for relationships between nutrient value nodes and nutrient description nodes.

What the query returns:

The query returns the connected nodes and the relationship between them, proving how the graph structure connects nutrient values with their descriptions.

```
neo4j@neo4j> MATCH (nv:NutVal)-[r:INSTANCE_OF]->(n:NutDes) RETURN nv,r,n LIMIT 2
0;
+-----+
| nv                                     | r               | n               |
+-----+-----+-----+
| (:NutVal {LastModified: "12/23/2013", SourceCode: 1, CNCode: 23129, NutrientValue: 17.22, ValueTypeCode: 1, PerUnit: "100g", NutrientCode: 203, DateAdded: "12/23/2013"}) | [:INSTANCE_OF] | (:NutDes {LastModified: "10/29/1997", NutrientDescriptionAbbrev: "Pro", NutrientDescription: "Protein", NutrientCode: 203, NutrientUnit: "g", DateAdded: "10/29/1997"}) |
+-----+-----+-----+
```

This assignment showed how graph databases can model relationships between interconnected datasets. Using Neo4j, nutrient description data and nutrient value data were imported from CSV files and transformed into graph nodes. Relationships were created between nodes using shared nutrient codes, allowing the graph database to stand for the connections between nutrient descriptions and nutrient values. The assignment also highlighted the differences between relational, document, and graph databases. While relational databases store data in tables and rely on joins to retrieve related data, graph databases store relationships directly within the database structure. This allows complex relationships to be analyzed more efficiently. Neo4j

proved to be an effective tool for interconnecting datasets. Graph databases provide a powerful approach for analyzing relationships between data points and offer advantages in applications where connections between data elements are critical.